Project Report

Student name: Ahmed Mahmoud Bahagat Mohamed Abdelmoula

Project name: Toyota full-stack web Application

Student number: 220303923

Date: June 2026

1. Project Overview

This report documents the development of a full-stack web application for Toyota Motor Corporation, built using ASP.NET Core MVC. The project started from a static HTML/CSS website and was transformed into a fully functional, database-driven web application with a complete backend, REST API, admin panel, and multiple interactive pages.

The project demonstrates the practical implementation of all key requirements outlined in the course, including C# programming, MVC architecture, Entity Framework Core, SQLite database, REST API design, authentication, and dependency injection.

2. Project Structure & Architecture

The project follows the standard ASP.NET Core MVC folder structure:

Folder / File	**Purpose**
Controllers/	All MVC and API Controllers (14 controllers total)
Models/	Database models: Vehicle, Category, Engineer, Manager, Offer, VehicleTrim, VehicleColor
Views/	Razor Views (.cshtml) for all pages (25+ views)
Data/AppDbContext.cs	EF Core DbContext with all DbSets and Seed Data
wwwroot/css/	CSS files for each page
wwwroot/js/	JavaScript files for dynamic behavior
wwwroot/images/	All vehicle and person images
Filters/AdminAuthFilter.cs	Session-based admin protection filter
Program.cs	App configuration, services, and middleware
appsettings.json	Connection string and app settings
toyota.db	SQLite database file

3. Requirements Implementation

Requirement 1 — C# Basics

Requirement: Variables, data types, conditions, loops, methods, classes, OOP, access modifiers, nullable types, collections, Async/Await.

Implementation:

- Used C# classes for all Models (Vehicle, Category, Engineer, Manager, Offer, VehicleTrim, VehicleColor)
- Applied access modifiers (public, private) throughout all Controllers and Models
- Used nullable types (string?, decimal?) in Models to handle optional fields like Badge and ImageUrl
- Used List<T> and ICollection<T> for relationships between models (e.g., Category has ICollection<Vehicle>)
- Applied async/await in all Controller actions that interact with the database (e.g., await _db.Vehicles.ToListAsync())
- Used LINQ methods like Where(), Select(), Include(), FirstOrDefaultAsync() for data querying

Requirement 2 — MVC Architecture (Model - View - Controller)

Requirement: Understanding and applying the MVC pattern — Model represents data, View is the UI, Controller handles requests.

Implementation:

- Models: 7 database models defined in Models/Models.cs representing all data entities
- Views: 25+ Razor Views (.cshtml) organized in folders by controller
- Controllers: 14 controllers handling all routes and business logic

Layer	Files	Count
Models	Vehicle, Category, Engineer, Manager, Offer, VehicleTrim, VehicleColor	7 Models
Views	Home, VehiclePage, Detail, Build, Shop, Service, Contact, Account, Admin, Saved, Offers	25+ Views
Controllers	Home, VehiclePage, Build, Shop, Service, Contact, Admin, Account, UserAccount, Saved, Offers Page + API Controllers	14 Controllers

Requirement 3 — ASP.NET Core MVC Project Setup

Requirement: Install .NET SDK, create project using CLI, understand folder structure, configure Program.cs and appsettings.json.

Implementation:

- Project created using: dotnet new mvc -n ToyotaProject
- Running on .NET 10.0 (the version installed on the development machine)
- Program.cs properly configured with all required services and middleware
- appsettings.json contains the SQLite connection string: Data Source=toyota.db
- Standard MVC folder structure maintained throughout the project
- Original HTML/CSS files moved to wwwroot/ folder as static assets

Requirement 4 — Models & Data Annotations

Requirement: Define C# model classes, use data annotations ([Required], [MaxLength]), understand primary keys and relationships.

Implementation:

- All models use [Required] and [MaxLength] data annotations for validation
- Primary keys defined using the Id property (EF Core convention)
- [Column(TypeName = "decimal(10,2)")] used on all price fields
- One-to-Many relationships: Category has many Vehicles; Vehicle belongs to one Category
- VehicleTrim and VehicleColor have foreign keys pointing to Vehicle

Model Relationships Structure:

Model	Key Fields	Relationships
Vehicle	Id, Name, Year, StartingMSRP, Badge, ImageUrl, CategoryId	Belongs to Category; Has many Trims & Colors
Category	Id, Name	Has many Vehicles
Engineer	Id, Name, Title, Bio, ImageUrl, Social Links	Independent
Manager	Id, Name, Role, ImageUrl	Independent
Offer	Id, Title, Description, IconClass	Independent
VehicleTrim	Id, Name, Price, FuelType, MPG, VehicleId	Belongs to Vehicle
VehicleColor	Id, Name, HexCode, VehicleId	Belongs to Vehicle

Requirement 5 — Entity Framework Core

Requirement: ORM concept, install EF Core, create DbContext, DbSet<T>, register in Program.cs, migrations, LINQ queries, CRUD operations.

Implementation:

- Packages installed: Microsoft.EntityFrameworkCore.Sqlite, Microsoft.EntityFrameworkCore.Tools, Microsoft.EntityFrameworkCore.Design
- AppDbContext class created in Data/AppDbContext.cs with all DbSets (Vehicles, Categories, Engineers, Managers, Offers, VehicleTrims, VehicleColors, AdminUsers)
- DbContext registered in Program.cs: builder.Services.AddDbContext<AppDbContext>(options => options.UseSqlite(...))
- Seed Data added in OnModelCreating(): 5 categories, 8 vehicles, 3 engineers, 3 managers, 6 offers, 17 trims, 24 colors
- Migrations created and applied: InitialCreate, AddTrimsAndColors
- LINQ used extensively: Include(), Where(), Select(), ToListAsync(), FindAsync(), FirstOrDefaultAsync(), CountAsync()
- Full CRUD operations implemented in Admin Panel for Vehicles, Engineers, and Managers

Requirement 6 — SQLite Database

Requirement: Understand SQLite, configure EF Core to use SQLite, database stored as .db file.

Implementation:

- SQLite configured in appsettings.json: "DefaultConnection": "Data Source=toyota.db"
- Database file toyota.db created automatically on first run via db.Database.Migrate() in Program.cs
- All tables created through EF Core migrations — no manual SQL written
- Database contains 9 tables: Categories, Vehicles, Engineers, Managers, Offers, VehicleTrims, VehicleColors, AdminUsers, __EFMigrationsHistory

Requirement 7 — Controllers

Requirement: Create controllers, action methods, difference between View and JSON responses, IActionResult, ViewBag, dependency injection.

Implementation:

- All controllers use IActionResult as return type for all actions
- View() for Razor Views, Ok() / NotFound() / BadRequest() / Json() for API responses
- ViewBag to pass additional data from Controller to View
- TempData for passing success messages across redirects
- RedirectToAction() for post-form redirects (PRG pattern)

MVC Controllers (return Views):

Controller	Actions	Description
HomeController	Index	Returns the main Home page

VehiclePageController	Index, Detail	Vehicles listing and detail pages
BuildController	Index	Build & Price page with trims and colors
ShopController	Index, Checkout, PlaceOrder, OrderConfirmation	Shopping and checkout flow
ServiceController	Index, History, Recalls, Schedule, ScheduleConfirmation	All service pages
ContactController	Index, Send, Confirmation	Contact form and confirmation
UserAccountController	Index, Login, Register, Dashboard, Logout	User account management
SavedController	Index, Toggle	Saved vehicles with session storage
OffersPageController	Index	Special offers page
AdminController	Index, Vehicles CRUD, Engineers CRUD, Managers CRUD	Full admin panel
AccountController	Login, Logout	Admin authentication

API Controllers (return JSON):

API Controller	Endpoints
VehiclesController	GET /api/vehicles, GET /api/vehicles?categoryId={id}, GET /api/vehicles/{id}, POST, PUT, DELETE
CategoriesController	GET /api/categories
EngineersController	GET /api/engineers, GET /api/engineers/{id}, POST, PUT, DELETE
ManagersController	GET /api/managers, GET /api/managers/{id}, POST, PUT, DELETE
OffersController	GET /api/offers

Requirement 8 — Routing

Requirement: Default route pattern, [Route] attributes, [HttpGet]/[HttpPost], route parameters.

Implementation:

- Default route configured in Program.cs: pattern: "{controller=Home}/{action=Index}/{id?}"
- API controllers use [Route("api/[controller]")] attribute
- [HttpGet], [HttpPost], [HttpPut], [HttpDelete] attributes used on all API actions
- [ValidateAntiForgeryToken] applied to all POST form actions

- Route parameters used for IDs: /VehiclePage/Detail/{id}, /Shop/Checkout/{id}, /Build/Index/{id}
- Query string parameters used for filtering: /api/vehicles?categoryId=1

Requirement 9 — HTTP Methods

Requirement: Understand and utilize proper HTTP methods for requests.

HTTP Method	Used For	Example in Project
GET	Read data, load pages	/api/vehicles — returns all vehicles as JSON
POST	Create new records, submit forms	/Admin/CreateVehicle — saves new vehicle to DB
PUT	Update existing records	/api/vehicles/{id} — updates vehicle via API
DELETE	Remove records	/Admin/DeleteVehicle — deletes vehicle from DB

Requirement 10 — REST API

Requirement: Design RESTful endpoints, return JSON, understand HTTP status codes.

Implementation:

- 5 API controllers with full RESTful design
- All API responses return proper JSON using Ok(), NotFound(), BadRequest(), CreatedAtAction()
- HTTP status codes used: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 404 Not Found
- API consumed by JavaScript Fetch on the frontend to dynamically load vehicles, engineers, managers, and offers
- Filtering supported via query parameters: GET /api/vehicles?categoryId=1

Requirement 11 — Program.cs Configuration

Requirement: Register services using builder.Services, set up middleware using app.Use, understand services vs middleware.

Implementation:

- Services registered: AddControllersWithViews() (MVC support), AddDbContext<AppDbContext>() (EF Core with SQLite), AddSession() (Session support)
- Middleware pipeline (in order): UseHttpsRedirection(), UseStaticFiles(), UseRouting(), UseSession(), UseAuthorization(), MapControllerRoute()
- Database automated migration on startup via db.Database.Migrate()

Requirement 12 — Admin Authentication (Session-Based)

Requirement: Use Session to track logged-in users, protect controllers, redirect unauthorized users.

Implementation:

- AdminAuthFilter class created in Filters/AdminAuthFilter.cs — inherits ActionFilterAttribute
- Filter checks HttpContext.Session.GetString("AdminLoggedIn") before each Admin action
- If session is empty, automatically redirects to /Account/Login
- [AdminAuthFilter] attribute applied to the entire AdminController class
- Login credentials: username = admin, password = admin123
- Session set on successful login: HttpContext.Session.SetString("AdminLoggedIn", "true")
- Logout clears all session data: HttpContext.Session.Clear()
- User Account also uses Session for regular users (separate from Admin session)

Requirement 13 — Dependency Injection (DI)

Requirement: Understand DI, inject DbContext into controllers via constructor.

Implementation:

- AppDbContext injected via constructor in all 14 controllers
- Pattern used consistently: private readonly AppDbContext _db; + constructor injection
- DI registered in Program.cs via builder.Services.AddDbContext<AppDbContext>()
- ASP.NET Core automatically resolves and injects the DbContext at runtime

4. Complete Pages & Features Summary

Page	URL	Features
Home	/	Hero, Shopping Tools, Dynamic Vehicles Grid, Offers, Stats, Pricing Table, Managers, Engineers, Footer
Vehicles	/VehiclePage	Sidebar category filter, Vehicle cards grid, Dynamic JS filtering
Vehicle Detail	/VehiclePage/Detail/{id}	Full vehicle info, specs, badge, Build & Shop buttons
Build & Price	/Build/Index/{id}	Trim selector, Color swatches, Live price update, Summary bar

Shop	/Shop	All vehicles with offers, Buy Now button
Checkout	/Shop/Checkout/{id}	Order summary, Personal info form, Payment form
Order Confirmation	/Shop/OrderConfirmation	Success message, next steps
Special Offers	/OffersPage	Offer banners, vehicles with offer tags
Support & Service	/Service	Service cards, Why Toyota section
Schedule Service	/Service/Schedule	Appointment booking form
Service History	/Service/History	Service records table
Recalls	/Service/Recalls	VIN checker, active recall cards
Contact	/Contact	Contact info, social links, message form
Account	/UserAccount	Login/Register modal, Dashboard
Saved Vehicles	/Saved	Saved vehicles grid with remove animation
Admin Login	/Account/Login	Secure admin login form
Admin Dashboard	/Admin	Stats cards
Admin Vehicles	/Admin/Vehicles	Full CRUD
Admin Engineers	/Admin/Engineers	Full CRUD
Admin Managers	/Admin/Managers	Full CRUD

5. Database Schema

Table	**Columns**	**Seed Records**
Categories	Id, Name	5 records
Vehicles	Id, Name, Year, StartingMSRP, AsShownPrice, Badge, ImageUrl, Description, DriveType, Engine, CategoryId	8 records
Engineers	Id, Name, Title, Bio, ImageUrl, Social Links	3 records
Managers	Id, Name, Role, ImageUrl	3 records
Offers	Id, Title, Description, IconClass	6 records
VehicleTrims	Id, VehicleId, Name, Price, FuelType, MPG, ImageUrl	17 records
VehicleColors	Id, VehicleId, Name, HexCode	24 records
AdminUsers	Id, Username, PasswordHash	1 record

6. Technologies Used

Technology	Version	Purpose
ASP.NET Core MVC	.NET 10.0	Web framework — Controllers, Views, Routing
Entity Framework Core	10.0.8	ORM — Database access, Migrations, LINQ
SQLite	via EF Core	Lightweight file-based database
C#	13.0	Backend programming language
HTML5 / CSS3	Custom	Frontend structure and styling
JavaScript (Vanilla)	ES6+	Dynamic behavior and Fetch API
Font Awesome 6	CDN	Icons throughout the website
Visual Studio Code	Latest	Development IDE

7. Conclusion

This project successfully demonstrates the implementation of a full-stack web application using ASP.NET Core MVC. Starting from a static HTML/CSS Toyota website, the project was transformed into a fully functional, database-driven application that covers all 13 requirements outlined in the course.

Key achievements:

- Clean MVC separation with 14 controllers, 7+ models, and 25+ views
- Fully functional REST API with 5 endpoints supporting GET, POST, PUT, and DELETE
- SQLite database with 9 tables, migrations, and pre-seeded data
- Session-based authentication for both Admin and regular User accounts
- Dynamic frontend behavior using JavaScript Fetch API without page reloads
- Complete Admin Panel for managing all website content
- Professional UI design consistent with the official Toyota website

The application is fully functional and runs locally using dotnet run, accessible at http://localhost:5113.
and the admin page opens from this link: http://localhost:5113/Account/Login.

Username: admin

Password: admin123

Project presentation:

https://drive.google.com/drive/folders/1TiEVcXO0NhXRXu8y4KwwGoTpOiGYGisk?usp=drive_link

Git hup link: https://github.com/AhmedBahgat-22/Toyota-web--Full-Stack.git

Project Demo Link: https://toyota-demo-app.onrender.com